5-MINUTE RECORDING TELEPROMPTER GUIDE

Slotbook Speaker & Narration Script

Synchronized with Candidate Cue & Teleprompter Beats

PRESENTER:
Tirumalesh Kommavarapu

TARGET DURATION:
Exactly 5 Minutes (0:00 - 5:00)

CAPTURES REQUIRED:
Screen + Webcam (Face) + Audio

TEST VERIFICATION:
71 of 71 Tests Passing (100%)

Teleprompter Timing & Section Overview

Timestamp	Teleprompter Beat	Screen Focus / Visual Cue
0:00 – 1:00	Beat 1: Show data model & explain double-book prevention	Data Model / `active_reservations` DDL
1:00 – 2:00	Beat 2: Booking lifecycle (hold, confirm, resched-ule)	State Machine & `clock_timestamp()`
2:00 – 3:00	Beat 3: Timezone logic: host zone vs guest zone display	Timezone Planes & 50m Stride Math
3:00 – 4:00	Beat 4: Optional extension built (Daily Cap / Outbox)	Partial Unique Index & Outbox Table
4:00 – 5:00	Beat 5: Show one test proving reliability claim	Terminal running `npm test` (71 passed)

—

Word-for-Word Presenter Narration

Minute 0:00 – 1:00: Data Model & Concurrency Protection

(Screen: Show Data Model / `active_reservations` schema)

[0:00] “Hello everyone. My name is Tirumalesh, and today I’m presenting Slotbook — a high-reliability appointment scheduling engine built with Next.js 14, TypeScript, and PostgreSQL 16.

[0:15] When building scheduling systems, the most dangerous architectural trap is relying on application-level checks like `SELECT count(*) WHERE start = time`. Under concurrent traffic, those checks suffer from time-of-check to time-of-use race conditions.

[0:30] In Slotbook, PostgreSQL is the single authoritative source of truth. We solved double-booking at the database storage engine level using the `btree_gist` extension on our `active_reservations` table.

[0:45] We enforce an exclusion constraint: `EXCLUDE USING gist (host_id WITH =, buffered_range WITH &&)`. Here, `buffered_range` is a continuous timestamp range spanning from the start time minus before-buffer to end time plus after-buffer. If two guests attempt to book the exact same slot concurrently, PostgreSQL physically rejects the second write with error code `23P01`, which our API translates into an immediate 409 Conflict.“

Minute 1:00 – 2:00: Booking Request Lifecycle & Postgres Correctness
(Screen: Show Booking State Machine: Hold -> Confirm -> Reschedule)

[1:00] “Now let’s trace a booking lifecycle: **Hold**, **Confirm**, and **Reschedule**.

[1:12] When a guest selects an available time slot, we don’t immediately confirm it. Instead, we issue a 5-minute hold lease in `active_reservations`. Crucially, this lease does not rely on application clocks or Redis TTLs; it authoritatively evaluates against PostgreSQL’s internal hardware clock: `clock_timestamp()`. If the user abandons the form, the slot is automatically reclaimed during the next sweep without locking the calendar hostage.

[1:32] When the guest enters their details and verifies their 6-digit email OTP, the reservation transitions to `CONFIRMED`.

[1:45] If the guest later needs to reschedule using their private self-service link, Slotbook performs an atomic transaction: it releases the existing reservation and locks the new target slot in a single commit. If the new slot has been taken by someone else in the interim, the entire transaction rolls back cleanly, leaving their original booking intact with zero data loss.“

Minute 2:00 – 3:00: Timezone Mathematics & Buffer Step Logic
(Screen: Show Timezone Architecture & Buffer Step Diagram)

[2:00] “Next, let’s look at how we handle timezones and Daylight Saving Time.

[2:10] We divide time into three strict planes: Storage, Host, and Guest. All timestamps in the database are stored canonically in UTC using `TIMESTAMPTZ`.

[2:22] When computing availability, our engine evaluates working hours against the host’s configured IANA timezone — for example, `Asia/Kolkata`. We apply recurring weekly rules, subtract date overrides and busy blocks, and project the available slots into the guest’s local browser timezone — for example, `America/New_York`.

[2:40] Notice our buffer cadence: candidate slot stepping strictly uses the full interval stride: $\text{Step} = \text{Duration} + \text{Buffer}_{\text{before}} + \text{Buffer}_{\text{after}}$. A 30-minute appointment with 10-minute before and 10-minute after buffers steps at 50-minute increments: 9:10, 10:00, 10:50. This mathematical formulation guarantees that hosts always receive their scheduled breaks.“

Minute 3:00 – 4:00: Optional Extensions Built
(Screen: Show Daily Limit Partial Index & Transactional Outbox Table)

[3:00] “Given the 48-hour timebox, we focused on extensions that reinforce operational reliability.

[3:12] The first extension we built is the **Daily Cap and Per-Guest Limit**. To prevent malicious users or bots from hoarding an entire day’s schedule, we created a PostgreSQL partial unique index on `(event_type_id, LOWER(guest_email), host_local_date)`. This enforces at the database level that a guest email can hold at most one active appointment per event type per local calendar day.

[3:32] The second extension is our **Transactional Outbox Pattern**. In naive applications, sending an email directly inside an HTTP handler introduces dual-write failure modes: if the SMTP server times out, the booking fails or hangs. In Slotbook, notifications are staged inside the same database transaction as the booking. An asynchronous background poller dispatches emails with exponential backoff and localized timestamps for both host and guest.“

Minute 4:00 – 5:00: Live Test Verification & Conclusion

(Screen: Switch to Terminal / Test Runner)

[4:00] “To empirically verify these reliability guarantees, we wrote comprehensive automated test suites.

[4:10] In `tests/concurrency.test.ts`, we simulate high-load race conditions: 10 asynchronous parallel requests hit the exact same host and time slot simultaneously.

(Execute or display `npm test` in the terminal)

[4:25] As you can see, all **71 tests pass across 7 suites**. In the concurrency test, exactly one transaction succeeds with HTTP 200 OK, while the other 9 receive an immediate HTTP 409 Conflict. Zero double-bookings occur.

[4:40] Slotbook is completely containerized. With a single command — `docker compose up --build` — PostgreSQL 16 initializes with `btree_gist`, migrations execute automatically, demo data seeds, and the Next.js server boots.

[4:55] Thank you for your time, and I look forward to your questions.“

—

Slotbook — 5-Minute Video Recording Guide

Candidate: Tirumalesh Kommavarapu | Repository: github.com/tiruk007/slotbooking